Vision Aided Navigation - Exercise 2

Student: Shay Morad

GitHub Repository: https://github.com/ShayMorad/3D-Computer-Vision-SLAM

(Note: Code is located on the main branch. Collaborator invites have been sent to course teachers).

Part 2.1: Matches in Rectified Stereo Images

- **The Pattern:** On a pair of rectified stereo images, a correct match will always have the exact same **Y-coordinate** (vertical pixel location) in both the left and right images. The deviation in the Y-axis should theoretically be zero.
- **The Cause:** "Rectification" is a mathematical transformation applied to the images that perfectly aligns the cameras so their optical axes are parallel and their image planes are perfectly coplanar. Therefore, the same physical feature is guaranteed to appear on the exact same horizontal scanline in both cameras. The only difference will be in the X-coordinate (horizontal shift), which is called *disparity* and is what allows us to calculate depth.
- **Percentage of matches deviating > 2 pixels:** 10.35%

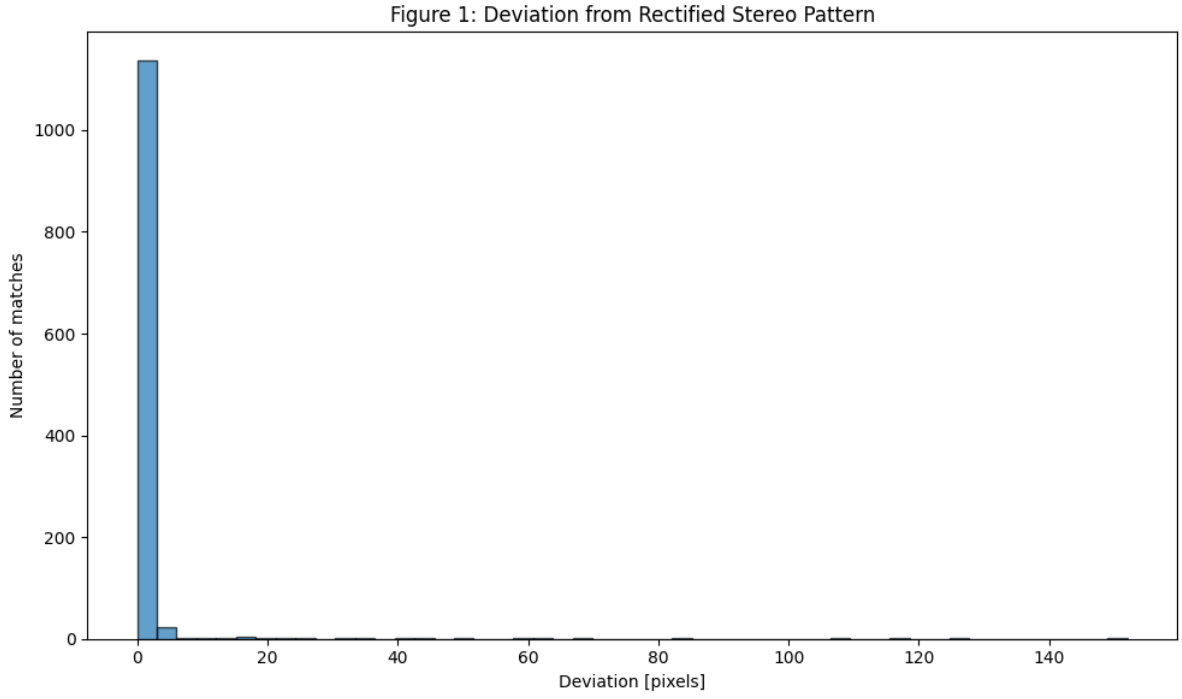


Figure 1: Histogram of number of matches per deviation category

Code Reference: ex2.py, Line 13

Part 2.2: Geometric Outlier Rejection Visualization

Using the 2-pixel vertical deviation threshold established in Part 2.1, the feature matches were filtered to separate geometrically consistent inliers from outliers.

- **Total matches discarded:** 123
- **Rejection Ratio (Uniform Assumption):** Assuming erroneous matches are distributed uniformly across the image, the probability of an error falling within our $\pm\mathbf{2}$ pixel (5-pixel total) vertical window is $\mathbf{5}/\boldsymbol{H}$, where $\boldsymbol{H} = \mathbf{376}$ is the image height. The expected rejection ratio is: $\mathbf{1} - \frac{\mathbf{5}}{\mathbf{376}} \approx \mathbf{98.67}\%$.
- **Expected wrongly accepted matches:** If 123 discarded matches represent 98.67% of total errors ($\boldsymbol{E}$), then $\boldsymbol{E} \approx \mathbf{124.6}$. The number of errors wrongly accepted is $\mathbf{124.6} - \mathbf{123} \approx \mathbf{2}$ matches.
- **Realism of Assumption:** This assumption is not realistic. In real-world driving scenes, erroneous matches are often caused by visually similar, repetitive structures (like lane markers or windows) that frequently lie on similar horizontal planes. Consequently, actual erroneous matches are more likely to bypass a vertical deviation filter than a uniform distribution would suggest. Therefore I would expect the actual number of accepted erroneous matches to be higher.

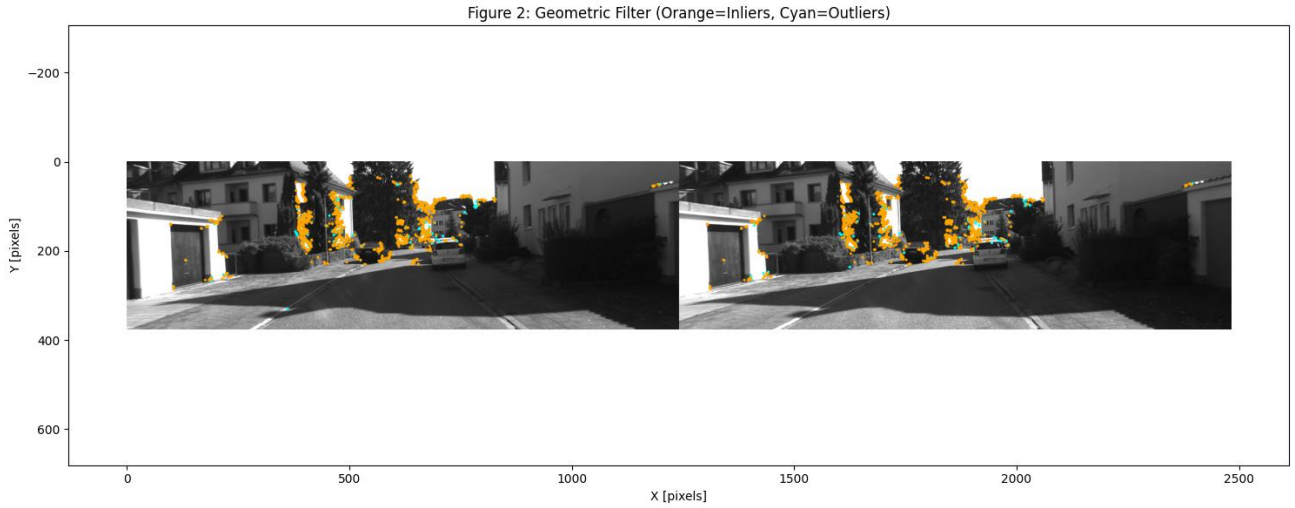


Figure 2: Feature matches categorized by the rectified stereo geometric policy

Code Reference: ex2.py, Line 37

Part 2.3: Stereo Triangulation

To project our 2D pixel coordinates into 3D space, a manual triangulation function was built as learnt in class.

- **Validation:** To verify the mathematical accuracy of the manual DLT implementation, the exact same pixel coordinates were processed using OpenCV's optimized cv2.triangulatePoints function. The calculated median distance between the manually derived 3D points and the OpenCV results was 1.35×10^{-13} meters:

```
Median distance between Manual and OpenCV 3D points: 1.3549e-13 meters
```

confirming the manual implementation is virtually identical to the library standard.

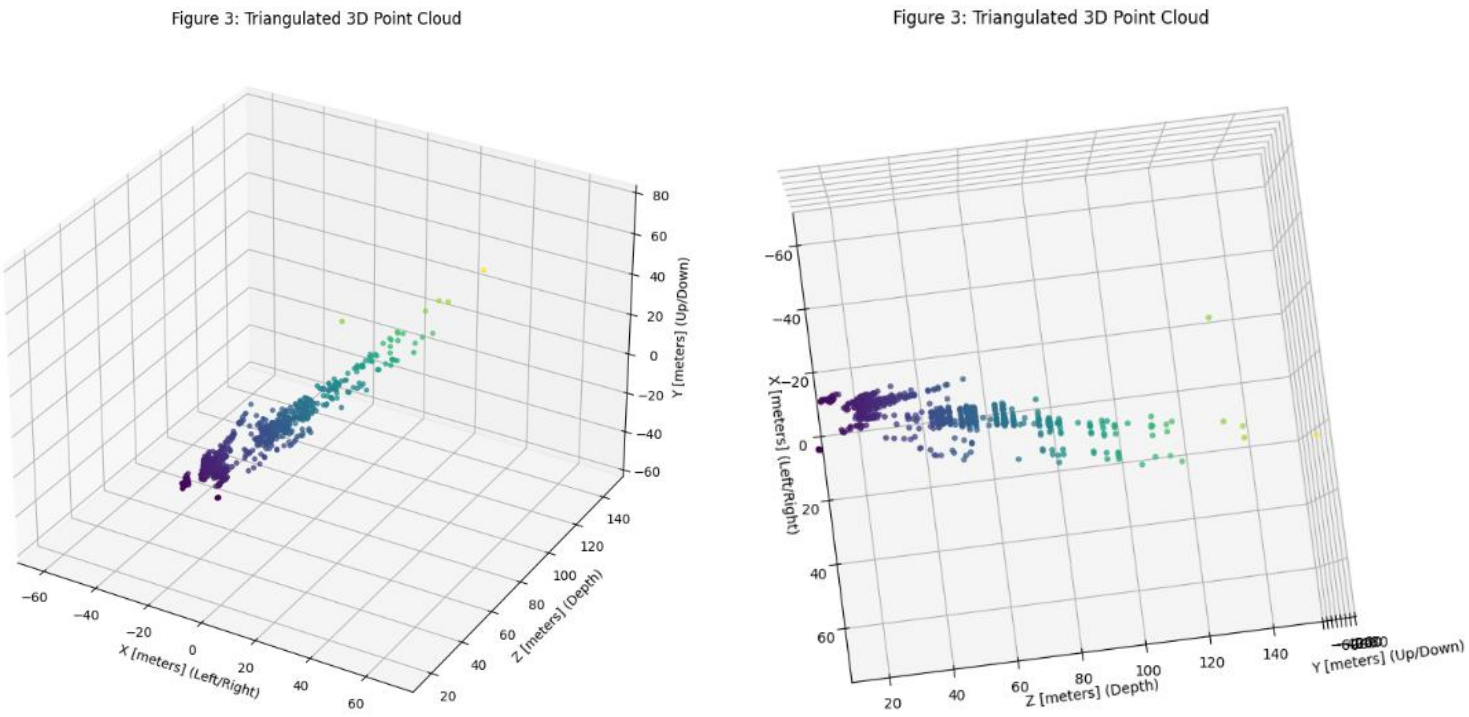


Figure 3: The reconstructed 3D point cloud.

Code Reference: ex2.py, Line 56

Part 2.4: Sequential Processing and Outlier Removal

The extraction, matching, and triangulation pipeline was run over the first three image pairs of the sequence.

- **Observation of Erroneous Points:** Yes, obviously erroneous 3D points are visible in the triangulated plots. While the vast majority of the 3D points form a dense, logical cluster representing the immediate physical structure of the road and nearby buildings, there are distinct outliers. These points project to impossible depths (Z>100 meters) and unrealistic vertical elevations (Y values placing them high in the sky or deep underground).
- **Reason for the Error:** These 3D errors are caused by false-positive feature matches of repetitive textures along the horizontal plane. Because the images are rectified, the geometric filter only checks if a match is horizontally aligned. If the ORB descriptor mistakenly matches two visually identical but physically different objects (e.g., repeating window frames or lane markers), the match passes the filter, but the horizontal disparity is artificially small. Because depth is inversely proportional to disparity, dividing by a near-zero disparity mathematically projects the triangulated 3D point toward infinity. Consequently, this massive depth calculation multiplies the vertical coordinate, causing the points to appear floating high in the air.
- **Relevant Criterion for Outlier Removal:** A highly relevant criterion for outlier removal is establishing a "Logical Physical Bounding Box." Since we know the physical context of the data (a camera mounted on a car driving on a street), we can define a region of valid 3D space based on real-world constraints. Any point triangulated outside these physical bounds should be rejected.
  For example:
  - $0 < Z < 80$ meters; rejecting points triangulated behind the car or too far away.
  - $-5 < Y < 5$ meters; rejecting points floating in the sky or beneath the road.
- **Edge Case Analysis:** Mathematical Singularity (Disparity of Zero) During the sequential processing of the frames, the system encountered a divide by zero runtime warning that produced infinite 3D coordinates, crashing the plotting sequence:

```
ValueError: Axis limits nan, inf cannot be infinite
```

  - **Cause:** This edge case occurs when the feature matcher incorrectly pairs two features with the exact same pixel coordinates in both the left and right images. In this scenario, the horizontal disparity is exactly zero. Because stereo depth is calculated as $Z = \frac{f \cdot B}{d}$, a disparity of zero mathematically projects the 3D depth to absolute infinity.

Code Reference: ex2.py, Line 112